Module 9 Week B — Stretch: GraphRAG Hybrid — Native Vector Index + Cypher Traversal

> ***Honors Track.*** *This stretch is for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently. Stretch counts toward Honors distinction; it is not required for program completion.*

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: complete the Module 9 Week B Applied Lab (entity-linking pipeline) — this stretch reuses the recipe-domain schema and the Bolt-driver discipline. Familiarity with the Module 8 vector-retrieval pattern (Weaviate) is also assumed; this stretch is the Neo4j-side counterpart of that primitive.

What you're building

A **GraphRAG hybrid retriever** over the Module 9 Week B recipe knowledge graph. You combine two retrieval substrates and fuse them at answer time:

- **Vector side** — Neo4j 5.x's native vector index over recipe descriptions, embedded with `sentence-transformers/all-MiniLM-L6-v2` (384-dim, the same model you used in Module 8).
- **Graph side** — a 1-hop Cypher traversal from each vector candidate to its cuisine, author, and ingredient list.

Neither substrate alone serves a query like "fragrant Sichuan noodle dishes." Vector retrieval finds semantically near recipes but does not know that Sichuan is a Chinese cuisine. Graph traversal can resolve the cuisine hierarchy via `[:SUBCLASS_OF*0..]` but does not know which descriptions are "fragrant." Fusing the two — at answer time, with a single ranking score — is the GraphRAG pattern. It is a first-class production retrieval pattern; the engine differs across Module 8 (Weaviate HNSW), Module 9 (Neo4j native vector), and Module 10 (deployment-layer retrieval), but the primitive is the same.

The learner deliverable is one function:

```
hybrid_retrieve(driver, embedder, query, k=10) -> list[dict]
```

assembled from three smaller building blocks under `retrieval/`.

Setup

> *Windows users. Commands below assume Bash/Zsh (macOS/Linux/WSL2/Git Bash). On PowerShell or CMD, activate your venv with `.venv\Scripts\Activate.ps1` (PowerShell) or `.venv\Scripts\activate.bat` (CMD), and confirm Docker Desktop is running with the WSL2 backend before `docker compose up -d`. Full cross-platform command tables are in the Pre-Work install plan.*

1. **Fork** github.com/LevelUp-Applied-AI/m9-s29b to your GitHub account (Fork button → owner = your account → Create fork).
2. **Enable Actions on your fork** (Actions tab → "I understand my workflows, go ahead and enable them"). One-time per fork.
3. **Clone your fork** and create the working branch:

   ```
   git clone https://github.com/<your-github-username>/m9-s29b.git
   cd m9-s29b
   git checkout -b stretch-9b-thu-graphrag
   ```

4. **Start Neo4j.** This stretch pins `neo4j:5.15-community` — the native vector index requires Neo4j 5.11 or later, and 5.15 is the minor release this build is verified against.

   ```
   docker compose up -d
   docker compose logs -f neo4j | head  # wait for "Started."
   ```

5. **Install Python dependencies.** Beyond the standard Bolt driver, this assignment adds `sentence-transformers` (≥2.7, <3). The first import downloads the MiniLM weights (~80 MB) and caches them locally; CI caches the same download via the HuggingFace hub cache.

   ```
   python -m venv .venv
   source .venv/bin/activate  # Windows: .venv\Scripts\activate
   pip install -r requirements.txt
   ```

6. **Load the fixture.** This step is course-provided. It loads the 50-recipe subgraph, embeds every recipe description with MiniLM, writes the resulting 384-float vector onto `:Recipe.embedding`, and creates the native vector index.

   ```
   python load_fixture.py
   ```

The vector index

`load_fixture.py` creates the index for you with the following Cypher:

```
CREATE VECTOR INDEX recipe_descriptions IF NOT EXISTS
  FOR (r:Recipe) ON r.embedding
  OPTIONS { indexConfig: {
    `vector.dimensions`: 384,
    `vector.similarity_function`: 'cosine'
  }};
```

- Index name: `recipe_descriptions` (your retrieval code must reference this exact name).
- Indexed property: `:Recipe.embedding`.
- Dimensions: **384** — the output size of `all-MiniLM-L6-v2`. Do not change this; if you embed with a different model you will get a dimension-mismatch error at query time.
- Similarity function: cosine, normalized by Neo4j into a `[0, 1]` score.

Confirm the index is online before you start implementing:

```
SHOW INDEXES YIELD name, type, state WHERE type = 'VECTOR' RETURN name, state
```

You should see `recipe_descriptions` with state `ONLINE`.

What you implement

Four small modules under `retrieval/`. The first three are independent building blocks; `hybrid.py` orchestrates them.

File	Function
`retrieval/vector_search.py`	`vector_candidates(driver, embedder, query, k)`
`retrieval/traversal.py`	`expand_context(driver, recipe_id)`
`retrieval/fuse.py`	`fuse(vector_results, contexts)`
`retrieval/hybrid.py`	`hybrid_retrieve(driver, embedder, query, k=10)`

Course-provided (do not modify): `retrieval/embed.py` (wraps the MiniLM embedder) and `load_fixture.py`.

Step 1 — Embed the query

In `vector_candidates`, embed the natural-language query into a 384-dim float vector using the course-provided `embed_text(embedder, query)` helper. The helper returns a Python list of floats — the shape Neo4j's vector procedure expects.

Step 2 — Pull vector candidates

Call the native vector procedure with the index name and the query vector as **parameters** — never string-interpolated:

```
CALL db.index.vector.queryNodes('recipe_descriptions', $k, $vector)
YIELD node, score
RETURN node.id AS recipe_id, node.name AS name, score
```

Bind `$k` and `$vector` via the Bolt driver's parameter map (`session.run(query, k=k, vector=qvec)`). Return a list of `{"recipe_id", "name", "score"}` dicts ordered by score DESC.

Step 3 — Expand graph context

For each vector candidate, fetch the 1-hop structural context. One Cypher query per candidate is fine for this assignment (50-recipe fixture, k≤10) — a batched version is part of the design discussion in `learner_notes.md`, not a requirement.

```
MATCH (r:Recipe {id: $id})
OPTIONAL MATCH (r)-[:OF_CUISINE]->(c:Cuisine)
OPTIONAL MATCH (r)<-[:BY_AUTHOR]-(a:Author)
OPTIONAL MATCH (r)-[:USES_INGREDIENT]->(i:Ingredient)
RETURN c.name AS cuisine,
       a.name AS author,
       collect(DISTINCT i.name) AS ingredients
```

Return a dict with three keys — `cuisine` (str or None), `author` (str or None), `ingredients` (sorted list of str). Sort the ingredient list alphabetically so output is deterministic.

Step 4 — Fuse vector + graph into one score

Use the documented fusion rule (also stated verbatim in the `fuse.py` docstring):

```
final_score = vector_score
            + STRUCTURAL_BOOST_PER_FIELD * (1 if cuisine     is non-empty else 0)
            + STRUCTURAL_BOOST_PER_FIELD * (1 if author      is non-empty else 0)
            + STRUCTURAL_BOOST_PER_FIELD * (1 if ingredients is non-empty else 0)
```

`STRUCTURAL_BOOST_PER_FIELD = 0.1` is course-provided in `fuse.py`. The rule combines a vector score in `[0, 1]` with up to three 0.1 boosts — keeping fused scores in a comparable range and rewarding candidates whose graph context is rich.

A "non-empty" field is one that is not `None`, and for the ingredient list also not `[]`.

Sort the fused list by `final_score` DESC and return it.

Step 5 — Orchestrate

`hybrid_retrieve` is the public entry point. It calls `vector_candidates`, builds a `{recipe_id: context_dict}` map by calling `expand_context` per candidate, calls `fuse`, and slices to the top-k.

The eval set

`data/eval_queries.json` holds **8 paraphrastic natural-language queries**, each with a small set of `gold_recipe_ids` — the recipes a high-quality retriever should surface in its top-10. Queries span multiple cuisines (Sichuan, Italian, Japanese, Mexican, North American) and use varied phrasing: synonyms, sensory descriptors, and category words. Exact-keyword matching alone is not enough.

The metric is **recall@10**, computed per query and macro-averaged across the 8 queries:

$$\text{recall\_at\_10}(\text{query}) = \frac{|\text{gold} \cap \text{top10}|}{|\text{gold}|}$$

The autograder asserts the macro-averaged recall meets the placeholder threshold **≥ 0.6**. The threshold is a placeholder pinned in Phase 6 against the reference implementation — your build will be calibrated against the same fixture.

Tests

```
pytest tests/ -v
```

The autograder covers:

- The `recipe_descriptions` vector index exists and is `ONLINE`; every `:Recipe` has a 384-dim embedding.
- `vector_candidates(...)` returns dicts with `recipe_id`, `name`, and a `score` in `[0, 1]`.
- `expand_context(...)` returns non-empty cuisine and ingredients for a known recipe.
- `fuse(...)` ranks results per the documented fusion rule.
- `hybrid_retrieve(...)` meets the recall@10 threshold across the 8 eval queries.
- **Fusion materially changes ranking from vector-alone.** The fixture ships 10 deliberately context-bare recipes (no cuisine, author, or ingredient edges — they exist only as embedded descriptions). A correct fusion implementation demotes them below context-rich recipes whose vector scores are close; an implementation that bypasses the structural boost (e.g., `return vector_results`) produces a fused ranking identical to vector-alone on every query and fails this gate. See `data/recipes_kg.cypher` for the bare-recipe list.
- Identity Discipline: no duplicate `:Entity.id` values.

Learner notes

`learner_notes.md` is read by the TA during review. Capture:

- One vector-index parameter you tuned (or chose not to tune) and why.
- Whether you fetched context per-candidate or batched, and why.
- Which of the 8 queries your pipeline retrieved well versus poorly, and what the failure mode looks like (wrong cuisine? lexically similar but topically distant?).
- One alternative fusion rule (e.g., weighted by ingredient overlap, or by hierarchical cuisine match via `[:SUBCLASS_OF*0..]`) and a quick hypothesis about which queries it would help most.

Common Pitfalls

- **Index name mismatch.** The procedure name argument must be exactly `'recipe_descriptions'` — typos here surface as `IndexNotFound` rather than `NameError`.
- **Embedding dimension mismatch.** MiniLM-L6-v2 emits 384 dimensions. If you swap in a different encoder mid-build (e.g., a 768-dim model) and re-embed only the query, Neo4j raises a dimension error at query time. The fixture's embeddings and the query embedding must come from the same model.
- **Embedding the query with the wrong model.** Even within MiniLM, calling `.encode()` on a different tokenizer or pooling configuration produces vectors that are technically 384-dim but live in a different space. Use the course-provided `embed_text(embedder, query)` so the query path matches the fixture path.
- **F-string interpolation into Cypher.** `f"CALL db.index.vector.queryNodes('recipe_descriptions', {k}, {vector})"` is wrong on two counts (injection surface and serialization of a Python list into Cypher text). Always use `$k` and `$vector` parameters and pass them to `session.run(query, k=..., vector=...)`.
- **Cuisine ancestor direction.** When you experiment with alternative fusion rules that consider the cuisine hierarchy, remember `(:Sichuan)-[:SUBCLASS_OF*0..]->(:Chinese)` is directional; the traversal pattern is `(start)-[:SUBCLASS_OF*0..]->(ancestor)`, not the reverse.
- **Unnormalized fusion weights.** If your alternative rule mixes a vector score in `[0, 1]` with an unbounded structural term (e.g., a raw ingredient-overlap count), the structural term either dominates the ranking entirely or vanishes against high vector scores. Normalize each component into the same range before weighting.

Submit

1. Commit your work and push the branch to your fork:

   ```
   git add .
   git commit -m "stretch 9b thu: GraphRAG hybrid retrieval"
   git push -u origin stretch-9b-thu-graphrag
   ```

2. **Open a Pull Request** ***within your fork*** — base `main`, compare `stretch-9b-thu-graphrag`, **base repository = your fork** (GitHub defaults to upstream `LevelUp-Applied-AI/m9-s29b` — change it to your fork).
3. Wait for the autograder to run green on your PR.
4. Paste your PR URL into **TalentLMS → Module 9 Week B → Stretch — GraphRAG Hybrid** to submit this assignment.